

실험 PC - Edge 분석 서버 REST API 명세

1. 목적

실험 PC에서 생성된 실험 결과 파일 bundle을 Edge 분석 서버로 전송하고, 전송 완료 후 보고서 생성을 요청하기 위한 REST API를 정의한다.

이 문서의 호출 주체는 XRD/TEM 실험 PC의 C# 전송 프로그램이고, 수신 주체는 Edge 분석 서버이다. C# 프로그램은 본 명세의 공통 작업 API를 호출하며, Spring Boot를 직접 호출하지 않는다. Edge 보고서 worker는 최종 보고서 ZIP을 공유 저장소에 게시하고 DB 전송 큐에 상대 경로를 등록한다.

2. 기본 정보

항목	값
API 버전	v1
Base URL	http://{edge-server}/api/v1
문자 인코딩	UTF-8
시간 형식	ISO 8601, KST 포함 (2026-06-13T14:30:25.123+09:00)
메타데이터 형식	application/json
파일 전송 형식	multipart/form-data

본 인터페이스는 사내 폐쇄망에서만 사용하며 별도의 애플리케이션 인증은 적용하지 않는다. Edge 서버의 API 포트는 방화벽 또는 서버 접근 정책을 통해 등록된 실험 PC에서만 접근할 수 있도록 제한한다.

환경별 기본 주소는 루트 공통 설정을 따른다.

환경	Base URL
개발	http://bhyoon.me:8000/api/v1
운영	http://{192.xxx.xxx.xxx:port}/api/v1

환경 선택과 주소 변경은 config/environments/development.env 및 config/environments/production.env 에서 관리한다.

3. 식별자

3.1 업무 복합 PK

하나의 실험 작업은 다음 네 필드의 조합으로 식별한다.

필드	타입	필수	설명	예시
requestNumber	string	Y	의뢰번호	REQ-2026-00123
experimentCode	string	Y	실험 종류 코드	XRD
equipmentCode	string	Y	실험장비 식별 코드	XRD-01
operatorId	string	Y	실험자 식별자	user01

동일 복합 PK로 재시험을 허용해야 하는 경우 PK 값을 임의로 변경하지 않고 서버가 내부 회차를 부여해 관리한다. 기본 명세에서는 활성 작업이 존재하는 동일 복합 PK의 중복 등록을 허용하지 않는다. 활성 작업이 종료된 뒤 동일 복합 PK로 재등록되면 최신 결과만 조회 대상으로 사용한다. 이전 결과에 대한 원상복구 또는 이력 조회 기능은 제공하지 않는다.

3.2 기술 식별자

Edge 서버는 작업 등록 시 UUID 형식의 jobId를 발급한다. 이후 파일 전송, 보고서 생성 요청 및 상태 조회는 jobId를 사용한다.

jobId는 업무 복합 PK를 대체하지 않으며 API 재시도와 내부 추적을 위한 기술 식별자이다.

4. Edge 저장 규칙

Edge 서버는 작업 등록 시 서버 시간을 기준으로 작업 폴더를 생성한다. 사용자 입력값은 파일시스템 안전 문자로 정규화해야 한다.

```
{storageRoot}/
  2026/06/13/
    20260613T143025123+0900_{jobId}/
      REQ-2026-00123_XRD_XRD-01_user01/
        input/
        processed/
        report/
        logs/
        manifest.json
```

- input: 실험 PC에서 전송한 원본 bundle
- processed: 전처리 및 분석 중간 산출물
- report: 최종 보고서와 결과 파일
- logs: 작업별 처리 로그
- manifest.json: PK, 파일 목록, 해시, 상태 및 시간 기록

- timestamp는 Edge 서버의 수신 시각을 사용한다.
- 사용자 관점에서는 동일 실험 재업로드 시 최신 회차 결과를 기본으로 표시할 수 있다.
- 본 명세 범위에서는 파일 원상복구 및 파일 형상관리 기능을 제공하지 않는다.

5. 전체 처리 순서

1. 실험 PC가 작업 등록 API를 호출한다.
2. Edge 서버가 jobId와 업로드 대상 경로를 반환한다.
3. 실험 PC가 파일을 하나씩 업로드한다.
4. 실험 PC가 파일 전송 완료 API를 호출한다.
5. Edge 서버가 파일 개수, 크기 및 SHA-256을 검증한다.
6. 실험 PC가 보고서 생성 API를 호출한다.
7. 실험 PC는 상태 조회 API로 처리 결과를 확인할 수 있다.

보고서 생성 요청은 파일 검증 완료 후에만 허용한다.

5.1 프로젝트별 적용 방식

프로젝트	파일 전송 주체	Edge 수신 API	보고서 생성/전달
XRD	C# 전송 프로그램	본 문서의 /api/v1/jobs, /files, /uploads/complete, /report	Edge worker가 보고서 ZIP 생성 후 공유 저장소 게시 및 DB 큐 등록
TEM	C# 전송 프로그램	본 문서의 /api/v1/jobs, /files, /uploads/complete, /report	Edge worker가 보고서 ZIP 생성 후 공유 저장소 게시 및 DB 큐 등록

XRD/TEM의 현재 웹 화면 API(/api/v1/xrd/analyze, /api/v1/tem/analyze)는 브라우저 기반 미리보기/보고서 생성용이다. C# 연동 계약은 위 공통 작업 API를 기준으로 한다.

5.2 XRD/TEM C# 클라이언트 권장 흐름

1. C# 프로그램이 GET /api/v1/requests?experimentType=XRD 또는 GET /api/v1/requests?experimentType=TEM 으로 의뢰 목록을 조회한다.
2. 사용자가 의뢰 항목을 선택한다.
3. 선택 항목의 requestNumber, experimentCode 와 C# 프로그램 설정에서 자동 입력된 equipmentCode, 사용자 로그인/선택값의 operatorId 로 POST /api/v1/jobs 를 호출한다.
4. C# 프로그램이 원본 파일 bundle을 POST /api/v1/jobs/{jobId}/files 로 전송한다.
5. 모든 파일 업로드가 성공하면 C# 프로그램이 GET /api/v1/jobs/{jobId}/files 로 서버에 등록된 최종 파일 목록을 다시 조회한다. 응답의 files 가 1개 이상일 때만 그 목록으로 POST /api/v1/jobs/{jobId}/uploads/complete 를 호출한다.
6. 사용자가 C# 프로그램의 보고서 생성 버튼을 누르면 C# 프로그램이 POST /api/v1/jobs/{jobId}/report 로 보고서 생성을 요청한다. XRD는 reportFormats: ["HTML"], TEM은 reportFormats: ["PPTX"] 를 보낸다.
7. C# 프로그램은 GET /api/v1/jobs/{jobId} 로 COMPLETED 또는 FAILED 상태를 확인한다. COMPLETED 는 Edge가 최종 ZIP을 공유 저장소에 게시하고 DB 전송 큐 등록까지 성공했다는 의미이다. LIMS 전송 결과는 별도 큐

상태로 관리한다.

프로젝트	보고서 생성 요청 reportFormats	공유 저장소 최종 ZIP의 핵심 산출물
XRD	["HTML"]	report.html, report.md, 선택 email_body.md
TEM	["PPTX"]	report.pptx, report.md, 선택 email_body.md

includeRawFiles=true를 보내면 위 산출물 외에 원본 bundle이 ZIP의 raw/ 경로에 포함된다. 기본값은 false이다.

현재 명세에서는 pk.experimentCode 값 하나가 Edge 작업 식별자와 Spring Boot 전달 필드 experimentCode로 함께 사용된다. LIMS 시험코드가 A23141, B54123처럼 장비 구분명(XRD, TEM)과 다르고, Edge processor는 장비 구분명으로 라우팅해야 하는 운영이면 C# 계약 확정 전에 다음 중 하나를 선택해야 한다.

- pk.experimentCode를 LIMS 시험코드로 보내고, 서버의 processor 설정을 해당 시험코드별로 맞춘다.
- 별도 필드(예: analysisType)를 추가해 Edge 내부 라우팅은 XRD / TEM, Spring Boot 전달은 LIMS experimentCode로 분리한다.

현 구현과 문서는 첫 번째 방식, 즉 단일 experimentCode 필드를 기준으로 한다.

5.3 XRD/TEM raw bundle 구성

C# 프로그램은 파일을 ZIP 하나로 묶어 보내는 방식보다, bundle 내부 파일을 풀어서 각 파일의 relativePath를 보존해 POST /api/v1/jobs/{jobId}/files로 전송하는 방식을 기본으로 한다. ZIP 파일 자체를 전송하려면 해당 실험 processor가 ZIP 해제를 지원하도록 별도 합의가 필요하다.

Edge 공통 업로드 API는 파일의 경로, 크기, SHA-256을 검증한다. 실제 보고서 생성 가능 여부는 실험별 processor가 입력 bundle을 해석해 processed/*.json을 만들 수 있는지에 따라 결정된다. 필수 파일이 부족하면 보고서 생성 상태 조회에서 ANALYSIS_RESULT_NOT_FOUND 또는 processor 오류가 반환될 수 있다.

C# 프로그램이 processed/*.json을 직접 생성해 업로드할 필요는 없다. 다만 Edge 서버에는 XRD/TEM 각각의 raw bundle을 읽어 구조화 분석 JSON을 생성하는 processor 명령이 설정되어 있어야 한다. processor 설정이 없거나 입력 경로 규칙이 맞지 않으면 raw 파일 전송과 uploads/complete가 성공해도 보고서 생성은 실패한다.

XRD bundle

XRD는 HTML 보고서를 만들기 위해 raw XRD 패턴 파일과 ICDD Card PDF가 필요하다. 피크 표와 보조 이미지는 선택 입력이다.

```
bundle-root/
  raw/
    {sampleName}.txt          # 필수. XRD 측정 패턴
  ICDD Card/
    *.pdf                     # 필수. 상매칭 후보 카드 PDF 1개 이상
  tables/
    *.xlsx | *.csv | *.tsv    # 선택. 피크 정보/정량 표
  images/
    *.png | *.jpg | *.jpeg    # 선택. 그래프/상매칭 보조 이미지
    *.webp | *.gif
```

지원 확장자:

구분	확장자	필수
raw 패턴	.txt, .dat, .xy, .asc	Y
ICDD Card	.pdf	Y
표	.xlsx, .csv, .tsv	N
이미지	.png, .jpg, .jpeg, .webp, .gif	N

권장 규칙:

- 한 작업은 기본적으로 하나의 XRD 보고서에 해당한다.
- raw 패턴 파일과 ICDD Card PDF가 모두 있어야 HTML 보고서 생성이 가능하다.
- raw 파일이 여러 개인 경우 어떤 파일을 대표 raw로 사용할지 processor 설정 또는 파일명 규칙을 사전에 설정해야 한다.
- C# 프로그램은 reportFormats: ["HTML"] 로 보고서 생성을 요청한다.

TEM bundle

TEM은 PPTX 보고서를 만들기 위해 tem, stem, report/reports, scale 중 하나 이상의 섹션 폴더에 분석 가능한 파일이 있어야 한다. 코팅층 두께 분석, EDS mapping, TEM/STEM 이미지 슬라이드 중 어떤 장표를 만들지에 따라 필요한 폴더가 달라진다.

```
bundle-root/
  tem/
    {sampleName}/
      *.tif | *.tiff | *.png
      *.jpg | *.jpeg | *.bmp | *.webp      # TEM 이미지
  stem/
    *.tif | *.tiff | *.png
    *.jpg | *.jpeg | *.bmp | *.webp      # STEM/BF-STEM 이미지
  report/
    *.docx                                # EDS MAP/Line/Point Word 보고서
    *.xlsx | *.xls | *.xlsm | *.xlsb
    *.csv | *.tsv                        # EDS raw spreadsheet
  reports/
    ...                                  # report와 동일하게 인식되는 별칭 폴더
  scale/
    {sampleName}/
      *.tif | *.tiff | *.png
      *.jpg | *.jpeg | *.bmp | *.webp      # 코팅층 두께 OCR 대상 이미지
```

지원 확장자:

구분	확장자	필수
TEM/STEM/scale 이미지	.tif, .tiff, .png, .jpg, .jpeg, .bmp, .webp	조건부
EDS Word 보고서	.docx	조건부
EDS raw 표	.xlsx, .xls, .xslm, .xlsb, .csv, .tsv	선택

권장 규칙:

- TEM 이미지 분석 장표가 필요하면 `tem/{sampleName}/` 아래에 시편별 이미지를 넣는다.
- STEM/BF-STEM 장표가 필요하면 `stem/` 아래에 이미지를 넣는다.
- EDS mapping/line/point 장표가 필요하면 `report/` 또는 `reports/` 아래에 Word 보고서를 넣는다.
spreadsheet는 원본 보존과 point 표 fallback에 사용될 수 있다.
- 코팅층 두께 OCR 장표가 필요하면 `scale/{sampleName}/` 아래에 두께 라벨이 포함된 이미지를 넣는다. `scale/` 바로 아래 이미지가 있으면 단일 시편 `Scale` 로 처리한다.
- C# 프로그램은 `reportFormats: ["PPTX"]` 로 보고서 생성을 요청한다.

6. 공통 헤더

헤더	필수	설명
X-Request-Id	Y	호출 추적용 UUID. 재시도 시 같은 값 사용
Idempotency-Key	POST/PUT/DELETE	중복 처리 방지 키. 재시도 시 같은 값 사용
Content-Type	본문 전송 시	JSON 또는 multipart 요청의 콘텐츠 타입
X-Client-Type	N	운영 기록의 클라이언트 유형. C# 프로그램은 <code>C#/.NET</code> 권장
X-Client-Name	N	운영 기록에 표시할 프로그램명. 예: <code>RIST XRD Uploader</code>
X-Client-Version	N	운영 기록에 표시할 프로그램 버전

X-Client-* 헤더를 생략해도 본 문서의 공통 작업 API는 실험 PC C# 전송 프로그램 호출로 분류한다. 다만 여러 버전의 전송 프로그램을 구분하고 장애를 추적하려면 세 헤더를 모든 요청에 일관되게 보내는 것을 권장한다.

Edge 운영 관리 화면(/operations)에는 C# 요청의 작업 PK, `jobId`, 실험 PC, 클라이언트 버전과 파일 상대 경로·원본명·크기·SHA-256이 기록된다. 파일 전송 완료 요청에는 최종 파일 수와 전체 크기도 기록된다. 업로드 검증 또는 처리에 실패한 요청은 동일한 정보가 오류 기록에도 연결된다.

사용 기록은 화면 접속과 실제 업무 수행을 구분한다. 보고서 생성 요청은 작업을 접수한 기록이고, 보고서 완료는 백그라운드 처리 후 보고서 패키지가 실제로 생성된 경우에만 별도 기록된다. 따라서 운영 화면의 기록 유형 필터에서 단순 화면 조회를 제외하고 실제 생성 완료 건수만 확인할 수 있다.

사용 기록 목록의 클라이언트 항목에는 C#/.NET 프로그램명, 실험 PC 이름과 접속 IP가 함께 표시된다. 리버스 프록시를 거치는 경우 X-Forwarded-For의 최초 접속 IP와 Edge 서버가 직접 연결받은 IP를 상세 화면에서 각각 확인할 수 있다.

7. API

7.1 작업 등록

```
POST /api/v1/jobs
Content-Type: application/json
Idempotency-Key: 771e92ae-d06d-42e3-b2c8-d1846619987c
```

요청

```
{
  "pk": {
    "requestNumber": "REQ-2026-00123",
    "experimentCode": "XRD",
    "equipmentCode": "XRD-01",
    "operatorId": "user01"
  },
  "sourcePc": {
    "hostName": "LAB-PC-XRD-01",
    "declaredIpAddress": "10.10.20.31",
    "clientVersion": "1.0.0"
  }
}
```

`equipmentCode` 는 XRD/TEM C# 전송 프로그램의 PC별 설정값으로 기본 입력한다. 실험 PC가 정해지면 장비도 사실상 정해지므로 일반 사용자가 매번 입력하지 않도록 하고, 필요 시 관리자 설정에서만 변경한다.

파일 수와 전체 크기는 이 시점에 선언하지 않는다. 파일 업로드·교체·삭제를 마친 뒤 `uploads/complete` 에서 최종 파일 목록과 함께 확정한다.

기존 PC 호환용 `bundle.fileCount` , `bundle.totalSizeBytes` 는 당분간 허용하지만 완료 검증 기준으로 사용하지 않는 deprecated 필드이다.

`sourcePc.declaredIpAddress` 는 실험 PC가 자체적으로 확인한 참고 정보이다. 접근 통제와 감사 로그에는 Edge 서버가 TCP 연결에서 확인한 원격 IP를 `observedRemoteIp` 로 별도 기록하고 이를 기준으로 사용한다.

성공 응답

새 작업을 만든 경우 201 Created 를 반환한다. 동일 업무 식별 기준 (`requestNumber` , `experimentCode` , `equipmentCode` , `operatorId`)의 활성 작업이 이미 있으면 새 작업을 만들지 않고 200 OK 와 기존 `jobId` 를 반환한다.

```
{
  "jobId": "e575b716-25d6-49c6-a7c0-3e2b7136fb2c",
  "status": "CREATED",
  "createdAt": "2026-06-13T14:30:25.123+09:00",
  "uploadExpiresAt": "2026-06-14T14:30:25.123+09:00",
  "reused": false
}
```

동일한 Idempotency-Key 와 동일한 요청 본문으로 작업 등록을 재호출하면 새 작업을 만들지 않고 기존 jobId 와 최초 성공 응답을 반환한다. 동일한 Idempotency-Key 로 다른 요청 본문을 보내면 409 Conflict 를 반환한다.

오류

- 400 Bad Request : PK 또는 sourcePc 정보 오류
- 409 Conflict : 동일 Idempotency-Key 의 요청 내용이 최초 요청과 다름
- 500 Internal Server Error : 폴더 또는 작업 생성 실패

7.2 파일 업로드

bundle 내 파일을 한 개씩 전송한다. 하위 폴더 구조가 필요한 경우 relativePath 에 bundle 루트 기준 상대 경로를 전달한다.

```
POST /api/v1/jobs/{jobId}/files
Content-Type: multipart/form-data
Idempotency-Key: {jobId}:{relativePath}:{sha256}
X-Client-Type: C#/.NET
X-Client-Name: RIST XRD Uploader
X-Client-Version: 1.0.0
```

multipart 필드

필드	타입	필수	설명
file	binary	Y	전송 파일
relativePath	string	Y	bundle 기준 상대 경로
sizeBytes	integer	Y	파일 크기
sha256	string	Y	소문자 64자리 SHA-256
lastModifiedAt	string	N	실험 PC 기준 수정 시각

relativePath 에는 절대 경로, .. , 드라이브 문자 및 null 문자를 허용하지 않는다.

성공 응답: 201 Created

```
{
  "fileId": "3fa4be94-2501-4cf7-b60d-616db3e628b8",
  "relativePath": "raw/Mix2.txt",
  "sizeBytes": 532481,
  "sha256": "4f0c93b48e4e8b95c2767a06c45020f3efc3bf45a172838807f90a8d8b43c26d",
  "status": "UPLOADED",
  "uploadedAt": "2026-06-13T14:31:02.410+09:00"
}
```

같은 Idempotency-Key 와 동일한 파일을 다시 전송하면 기존 성공 결과를 반환한다. 경로는 같지만 해시가 다르면 409 Conflict 를 반환한다. 단, 해당 파일을 DELETE 로 삭제한 뒤 같은 파일을 다시 업로드하는 경우에는 같은 Idempotency-Key 를 사용하더라도 현재 파일 목록에 새로 등록된다.

파일 교체·조회·삭제

```
PUT /api/v1/jobs/{jobId}/files/{relativePath}
GET /api/v1/jobs/{jobId}/files
DELETE /api/v1/jobs/{jobId}/files/{relativePath}
```

- PUT 은 같은 relativePath 의 파일을 새 파일로 교체한다. multipart 필드는 POST 와 같되 경로는 URL에서 받으므로 relativePath 를 보내지 않는다.
- GET 은 현재 업로드된 파일의 경로, 크기, SHA-256, 업로드 시각을 반환한다.
- DELETE 는 Edge 저장 파일과 메타데이터를 함께 삭제한다.
- 삭제 후 동일 relativePath /동일 SHA-256 파일을 다시 업로드하면 새 파일 레코드로 등록되며, 이후 GET 파일 목록에 포함된다.
- 세 작업은 CREATED , UPLOADING 상태에서만 허용한다. FILES_VERIFIED 이후 입력 파일은 불변이며 변경 요청은 409 Conflict 를 반환한다.

GET /api/v1/jobs/{jobId}/files 응답 예시는 다음과 같다.

```
{
  "jobId": "e575b716-25d6-49c6-a7c0-3e2b7136fb2c",
  "files": [
    {
      "fileId": "3fa4be94-2501-4cf7-b60d-616db3e628b8",
      "relativePath": "raw/Mix2.txt",
      "sizeBytes": 532481,
      "sha256": "4f0c93b48e4e8b95c2767a06c45020f3efc3bf45a172838807f90a8d8b43c26d",
      "status": "UPLOADED",
      "uploadedAt": "2026-06-13T14:31:02.410+09:00"
    }
  ]
}
```

오류

- 400 Bad Request : 잘못된 상대 경로 또는 메타데이터
- 404 Not Found : 존재하지 않는 jobId
- 409 Conflict : 이미 다른 내용으로 등록된 경로
- 410 Gone : 업로드 유효기간 만료
- 413 Payload Too Large : 서버 제한을 초과한 파일
- 422 Unprocessable Entity : 크기 또는 SHA-256 불일치

7.3 파일 전송 완료

실험 PC가 모든 파일 업로드·교체·삭제를 마친 뒤 호출한다. 이 요청은 이번 분석에 사용할 최종 입력 파일 집합을 확정하는 선언이며, Edge 서버는 실제 업로드 파일의 개수, 전체 크기, 개별 해시를 검증한다.

완료 요청 본문은 사용자가 선택한 로컬 파일 목록이 아니라, 모든 업로드가 끝난 뒤 GET /api/v1/jobs/{jobId}/files 에서 받은 **서버 등록 파일 목록**으로 구성하는 것을 권장한다. 다음 조건을 하나라도 만족하지 않으면 C# 프로그램은 완료 API를 호출하지 말고 사용자에게 업로드 미완료를 표시한다.

- files.Count >= 1
- fileCount == files.Count
- totalSizeBytes == files.Sum(file => file.sizeBytes)
- 모든 파일에 relativePath, sizeBytes, 소문자 64자리 sha256 이 있음
- 전송을 시작한 모든 파일의 업로드 응답이 성공이고 서버 파일 목록에도 존재함

특히 fileCount: 0, files: [] 인 빈 bundle은 허용하지 않으며 서버는 422 Unprocessable Entity 를 반환한다. 빈 bundle 검증을 의도한 부정 테스트가 아니라면 이 요청을 보내는 것은 C# 클라이언트 구현 오류이다.

```
POST /api/v1/jobs/{jobId}/uploads/complete
Content-Type: application/json
Idempotency-Key: {jobId}:uploads-complete
```

요청

```
{
  "fileCount": 3,
  "totalSizeBytes": 8392401,
  "files": [
    {
      "relativePath": "raw/Mix2.txt",
      "sizeBytes": 532481,
      "sha256": "4f0c93b48e4e8b95c2767a06c45020f3efc3bf45a172838807f90a8d8b43c26d"
    },
    {
      "relativePath": "cards/Ti02-00-021-1276.pdf",
      "sizeBytes": 3926040,
      "sha256": "3500ca035e6d223cb2cfb7b5ed1e4cb75268ee7ae38c50be50e72d6e282fd204"
    },
    {
      "relativePath": "cards/Ti02-00-064-0863.pdf",
      "sizeBytes": 3933880,
      "sha256": "67c7fdb4b554d171373b45a1db42319a425dc543cd1080698af5ec11df742032"
    }
  ]
}
```

`files`에는 실제 bundle의 전체 파일을 전달한다. `fileCount`는 `files` 배열 길이와 같아야 하고, `totalSizeBytes`는 파일 크기의 합계와 같아야 한다.

C# 완료 요청 구성 예시

아래 예시는 서버 파일 목록 조회 결과를 기준으로 완료 본문을 만드는 핵심 검증 로직이다. `JobFilesResponse.Files`에는 7.2절의 파일 목록 응답을 역직렬화한 값이 들어 있다고 가정한다.

```

var serverFiles = jobFilesResponse.Files;
if (serverFiles is null || serverFiles.Count == 0)
{
    throw new InvalidOperationException(
        "서버에 등록된 파일이 없어 uploads/complete를 호출할 수 없습니다.");
}

var completeBody = new
{
    fileCount = serverFiles.Count,
    totalSizeBytes = serverFiles.Sum(file => file.SizeBytes),
    files = serverFiles.Select(file => new
    {
        relativePath = file.RelativePath,
        sizeBytes = file.SizeBytes,
        sha256 = file.Sha256.ToLowerInvariant()
    }).ToArray()
};

using var completeRequest = new HttpRequestMessage(
    HttpMethod.Post,
    $"api/v1/jobs/{jobId}/uploads/complete");
completeRequest.Headers.Add("X-Request-Id", requestId);
completeRequest.Headers.Add("Idempotency-Key", $"{jobId}:uploads-complete");
completeRequest.Content = JsonContent.Create(completeBody);

using var completeResponse = await httpClient.SendAsync(completeRequest);
// 400/409/410/422는 요청을 수정하기 전에는 자동 재시도하지 않는다.
completeResponse.EnsureSuccessStatusCode();

```

동일한 `Idempotency-Key` 와 동일한 요청 본문으로 재호출하면 기존 `FILES_VERIFIED` 응답을 반환한다. 이미 검증이 끝난 작업에 다른 파일 목록을 보내면 `409 Conflict` 를 반환한다.

성공 응답: 200 OK

```

{
  "jobId": "e575b716-25d6-49c6-a7c0-3e2b7136fb2c",
  "status": "FILES_VERIFIED",
  "verifiedFileCount": 3,
  "verifiedAt": "2026-06-13T14:31:10.032+09:00"
}

```

오류

- `409 Conflict`: 아직 업로드 중이거나 이미 보고서 생성이 시작됨

- 410 Gone : 업로드 유효기간 만료
- 422 Unprocessable Entity : 빈 `files`, `fileCount < 1`, 누락 파일, 개수, 크기 또는 해시 불일치

7.4 보고서 생성 요청

```
POST /api/v1/jobs/{jobId}/report
Content-Type: application/json
Idempotency-Key: {jobId}:generate-report
```

요청

```
{
  "requestedAt": "2026-06-13T14:31:15.000+09:00",
  "options": {
    "reportFormats": ["HTML"],
    "includeRawFiles": false
  }
}
```

`options` 는 선택 필드이다. `reportFormat` 은 기존 실험 PC 호환을 위해 유지하며, 새 구현은 `reportFormats` 배열을 사용한다. 두 필드를 함께 보내면 `reportFormats` 가 우선한다.

- `reportFormats` 지원값: PPTX, PDF, HTML (중복 불가)
- 두 형식을 모두 생략한 경우 기본값: PPTX
- `includeRawFiles` 를 생략한 경우 기본값: `false`
- `includeRawFiles=true` 이면 Edge가 공유 저장소에 게시하는 최종 ZIP에 원본 bundle을 `raw/` 경로로 함께 넣는다. `false` 여도 원본은 Edge `input/` 에 보관하며 ZIP에만 포함하지 않는다.

XRD/TEM C# 프로그램은 다음 형식을 사용한다.

XRD HTML 보고서:

```
{
  "requestedAt": "2026-06-13T14:31:15.000+09:00",
  "options": {
    "reportFormats": ["HTML"],
    "includeRawFiles": false
  }
}
```

TEM PPT 보고서:

```
{
  "requestedAt": "2026-06-13T14:31:15.000+09:00",
  "options": {
    "reportFormats": ["PPTX"],
    "includeRawFiles": false
  }
}
```

성공 응답: 202 Accepted

```
{
  "jobId": "e575b716-25d6-49c6-a7c0-3e2b7136fb2c",
  "status": "QUEUED",
  "acceptedAt": "2026-06-13T14:31:15.120+09:00"
}
```

이 API는 비동기 처리 시작만 보장한다. 최종 완료 여부는 상태 조회 응답으로 확인한다.

worker는 요청한 사용자용 report.pdf, report.pptx, report.html 중 해당 형식만 렌더링하고, 공통 Markdown 요약인 report.md 를 만든 뒤 {jobRoot}/report/report-package.zip 으로 묶는다. 분석 결과 JSON, LLM 요청/응답 JSON, 내부 report.json 은 Edge 내부 처리용이며 공유 저장소 최종 ZIP에는 포함하지 않는다. DB 큐 및 공유 저장소 계약은 [EDGE SPRING BOOT API.md](#) 를 따른다.

오류

- 404 Not Found: 존재하지 않는 jobId
- 409 Conflict: 파일 검증 미완료 또는 현재 상태에서 실행 불가
- 422 Unprocessable Entity: 해당 실험/장비 processor를 찾을 수 없음

7.5 작업 상태 조회

```
GET /api/v1/jobs/{jobId}
X-Request-Id: 771e92ae-d06d-42e3-b2c8-d1846619987c
```

성공 응답: 200 OK

```
{
  "jobId": "e575b716-25d6-49c6-a7c0-3e2b7136fb2c",
  "pk": {
    "requestNumber": "REQ-2026-00123",
    "experimentCode": "XRD",
    "equipmentCode": "XRD-01",
    "operatorId": "user01"
  },
  "status": "PROCESSING",
  "progress": 65,
  "createdAt": "2026-06-13T14:30:25.123+09:00",
  "processingStartedAt": "2026-06-13T14:31:16.003+09:00",
  "completedAt": null,
  "error": null
}
```

실패한 경우:

```
{
  "jobId": "e575b716-25d6-49c6-a7c0-3e2b7136fb2c",
  "status": "FAILED",
  "progress": 90,
  "error": {
    "code": "REPORT_QUEUE_REGISTRATION_FAILED",
    "message": "보고서 전송 큐 등록에 실패했습니다.",
    "retryable": true
  }
}
```

7.6 의뢰 번호 목록 조회

```
GET /api/v1/requests?page=1&pageSize=50&experimentType=XRD&includeCompleted=false
X-Request-Id: 771e92ae-d06d-42e3-b2c8-d1846619987c
```

Edge 로컬 MariaDB의 `lims_req_ax_search` 테이블을 조회해 LIMS 의뢰/시료/시험 항목 목록을 반환한다. 이 API는 조회용이며 작업을 생성하지 않는다. 사용자는 목록에서 의뢰번호와 시험 항목을 선택한 뒤 `POST /api/v1/jobs` 를 호출한다. `experimentType` 은 선택 파라미터이며, 화면별 조회 범위를 제한할 때 사용한다. XRD/TEM C# 전송 프로그램은 각각 XRD, TEM 을 전달한다.

- `experimentType` 필터는 `test_mtd_code`, `test_mtd_name` 기준으로 적용한다. 예를 들어 `test_mtd_name` 에 XRD, TEM, 전자현미경처럼 시험/장비명이 명시되어 있으면 해당 화면의 시험 항목만 조회한다.
- 파라미터를 생략하면 완료되지 않은 전체 시험 항목을 조회한다.
- `includeCompleted` 기본값은 `false` 이다. `req_state_name` 에 완료 가 포함된 시험 항목은 기본 조회에서 제외한다. 완료 건을 포함해야 하는 관리 화면은 `includeCompleted=true` 를 명시한다.

```
{
  "page": 1,
  "pageSize": 50,
  "experimentType": "XRD",
  "items": [
    {
      "requestResultNo": 270846,
      "requestNumber": "2025M01309",
      "requestDate": "2026-05-19",
      "requestState": 8,
      "requestStateName": "접수",
      "requestTypeNo": 123,
      "requestTypeCode": "M1",
      "requestTypeName": "분석의뢰",
      "projectCode": "P-XRD",
      "customerRequestName": "결정상 분석",
      "customerNo": 9,
      "customerName": "RIST 고객",
      "requestUserNo": 102,
      "requestUserName": "김의뢰",
      "sampleResultNo": 458465,
      "sampleName": "XRD 시료",
      "sampleState": 3,
      "testMethodResultNo": 485993,
      "testMethodNo": 3912,
      "testMethodCode": "A23141",
      "testMethodName": "XRD 데이터 해석",
      "testState": 2,
      "testChargerName": "이현재",
      "outputOrder": 1,
      "syncedAt": "2026-07-03 18:10:35",
      "experimentCode": "A23141",
      "experimentName": "XRD 데이터 해석"
    }
  ]
}
```

experimentCode 와 experimentName 은 실험 PC 선택 화면 호환을 위한 별칭이며, 각각 testMethodCode , testMethodName 과 같은 값이다. page 는 1 이상, pageSize 는 1~200이다. 응답에는 의뢰번호 외에도 의뢰일, 의뢰상태, 의뢰명, 시료명, 시험명, 의뢰자, 시험담당자 등 화면에서 상세 확인에 필요한 LIMS 조회 정보가 포함된다.

XRD/TEM C# 프로그램은 이 응답에서 requestNumber 와 experimentCode 를 선택값으로 사용한다. equipmentCode 는 실험 PC별 프로그램 설정값을 기본으로 자동 입력하고, operatorId 는 사용자 로그인 또는 선택값으로 채워 작업 등록 API에 전달한다. testChargerName 은 화면 표시용 담당자명이며, operatorId 를 대체하는 고유 식별자로 사용하지 않는다.

8. 작업 상태

상태	설명
CREATED	작업과 저장 폴더 생성
UPLOADING	파일 수신 중
UPLOAD_EXPIRED	업로드 유효기간이 지나 작업 등록이 만료됨
FILES_VERIFIED	파일 수신 및 무결성 검증 완료
QUEUED	보고서 생성 대기
PROCESSING	전처리, 분석 또는 보고서 생성 중
COMPLETED	최종 ZIP 생성, 공유 저장소 확정 및 DB 전송 큐 등록 완료
FAILED	복구되지 않은 오류로 작업 실패

상태는 이전 단계로 되돌리지 않는다.

COMPLETED 는 Edge 작업 완료를 뜻하며 LIMS 전송 완료를 뜻하지 않는다. Spring Boot는 `report_transfers` 큐를 스케줄링해 공유 저장소의 ZIP을 직접 읽고 LIMS에 전달한다. LIMS 전달 상태는 `PENDING`, `PROCESSING`, `RETRY_WAIT`, `COMPLETED`, `FAILED`, `CANCELLED`로 별도 관리하며, 상세 계약과 스키마는 `EDGE_SPRING_BOOT_API.md` 및 `edge_api_server/deploy/mariadb_report_queue.sql`을 따른다.

`uploadExpiresAt` 이후 검증이 완료되지 않은 작업은 `UPLOAD_EXPIRED`로 전환한다. 만료된 작업에 대한 파일 업로드와 업로드 완료 요청은 `410 Gone`을 반환한다. 실험 PC는 새 `Idempotency-Key`로 작업 등록부터 다시 시작해야 하며, 만료된 `jobId`는 재사용하지 않는다.

9. 공통 오류 응답

```
{
  "timestamp": "2026-06-13T14:31:10.032+09:00",
  "status": 422,
  "code": "FILE_HASH_MISMATCH",
  "message": "파일 SHA-256이 요청값과 일치하지 않습니다.",
  "requestId": "dd5c9c31-bec3-40cd-96aa-2aec9dd6fb11",
  "jobId": "e575b716-25d6-49c6-a7c0-3e2b7136fb2c",
  "retryable": true
}
```

오류가 Edge 통합 오류 보관소에 기록되면 응답 헤더에 `X-Error-Event-Id`가 포함되고, `X-Error-Comment-Url`에는 고객 코멘트 입력 주소가 포함된다. JSON 오류 본문에도 같은 값을 `errorEventId`, `errorFeedbackUrl`로 제공한다. 운영자는 `/errors` 화면에서 이 ID로 로그와 실패 당시 보관 파일을 조회할 수 있다. 고객 또는 실험 PC 프로그램은 다음 API로 재현 상황을 추가할 수 있다.

```
POST /api/v1/errors/{errorEventId}/comments
Content-Type: application/json

{
  "author": "실험실 PC 사용자",
  "content": "Windows 11에서 같은 PDF를 업로드하면 반복해서 발생합니다."
}
```

브라우저에서는 `/error-feedback/{errorEventId}` 를 열어 같은 내용을 입력한다. 오류 관리 화면과 고객 입력 화면은 내부 운영망에서만 접근을 허용한다.

10. 재시도 및 타임아웃

- 연결 타임아웃: 5초 권장
- 파일 업로드 읽기 타임아웃: 파일 크기에 따라 5분 이상
- 일반 API 읽기 타임아웃: 30초
- 재시도 대상: 네트워크 오류, 408, 429, 502, 503, 504
- 재시도 간격: 1초, 2초, 4초, 8초, 16초의 지수 백오프
- 400, 409, 410, 422 는 요청 수정 없이 자동 재시도하지 않는다.
- POST 재시도 시 최초 요청과 같은 Idempotency-Key 를 사용한다.

11. 파일 및 보안 규칙

- 파일명과 상대 경로는 원본을 보존하되 서버에서 경로 탐색 공격을 차단한다.
- 실행 파일과 스크립트의 허용 여부는 장비별 allowlist로 제한한다.
- 수신 파일은 보고서 처리 전에 SHA-256을 검증한다. 악성코드 검사는 Edge 운영 환경의 별도 보안 정책으로 적용한다.
- Edge API 포트는 등록된 실험 PC의 고정 IP에서만 접근하도록 제한한다.
- API 로그에는 PK, jobId, 요청 시각, 호출 PC 및 결과 코드가 포함되어야 한다.
- 원본 실험 파일과 보고서의 보존 및 삭제 기간은 운영 정책으로 별도 관리한다.